27. Dynamic Programming Patterns

27. Dynamic Programming Patterns

Phase: Phase 4 — Algorithms and data structures
Estimated time: 2 days
Slug: `27-dynamic-programming`

Learning objectives

- Identify overlapping subproblems
- Write bottom-up DP tables
- Memoize top-down recursion
- Apply classic patterns (knapsack, LIS, paths)

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Dynamic programming turns exponential search into polynomial tables. In infra, DP thinking shows up in scheduling and path problems.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Identify overlapping subproblems + optimal substructure.
- Bottom-up tables vs top-down memo maps.
- Define state clearly: dp[i] meaning in one sentence.
- Watch index off-by-ones and base cases.
- Reconstruct solutions by storing choices, not only values.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-dynamic && cd /tmp/gw-dynamic
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func Climb(n int) int {
    if n <= 2 {
        return n
    }
    a, b := 1, 2
    for i := 3; i <= n; i++ {
        a, b = b, a+b
    }
    return b
}

func main() { fmt.Println(Climb(5)) }
```

Run it:

```
go run .
```

Expected output:

```
8
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Climbing stairs bottom-up.
2. Memoize a recursive solution with a map.
3. Write the dp state meaning as a comment first.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Idea

DP stores answers to subproblems to avoid recomputation.

```
// Climb stairs: ways(n) = ways(n-1)+ways(n-2)
func Climb(n int) int {
    if n < 2 { return 1 }
    a, b := 1, 1
    for i := 2; i <= n; i++ {
        a, b = b, a+b
    }
    return b
}
```

Pattern checklist

1. Define state clearly (`dp[i]` means …)
2. Write recurrence
3. Determine base cases
4. Choose iteration order
5. Optimize space if only last states needed

Infra relevance

DP appears in scheduling, rate allocation, tokenization costs, and alignment-like problems — not only interview questions.

Failure modes to watch while coding

- Memoizing without clear state definition
- Off-by-one in table sizes
- Exponential backtracking labeled as DP

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- functools.lru_cache ≈ map memo in Go.
- Same recurrence, different syntax.
- Be explicit about mutation of DP slices.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

If you cannot name the state, you are not ready to code the loop.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Dynamic Programming Patterns” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked

example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Memoizing without clear state definition
- Off-by-one in table sizes
- Exponential backtracking labeled as DP

Exercises

Work in `exercises/27-dynamic-programming/`. Read that folder's `README.md` before you code.

What you will build

Implement the DP function (e.g. climb/knapsack-style) to match tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/27-dynamic-programming/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Overlapping subproblems meaning?
2. Top-down vs bottom-up?
3. When reduce DP space?

Further reading

- https://en.wikipedia.org/wiki/Dynamic_programming